

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA



PUCP

**Ingeniería de las
Telecomunicaciones**

Curso: Ingeniería de redes cloud

Proyecto del curso

ÍNDICE

1. Introducción	4
2. Arquitectura.....	4
3. Descripción de módulos.....	5
3.1 Módulo User Interface	5
3.1.1 Conexión User Interface - Slice Manager	5
3.2 Módulo Slice Manager.....	6
3.2.1 Conexión Slice Manager - VM Placement.....	7
3.3 Módulo de VM Placement	8
3.3.1 Coordinación Slice Manager - VM Placement.....	9
3.4 Módulo de Drivers	10
3.4 Módulo de Monitoreo.....	10
3.5 Módulo de Persistence & Logging	11
4. Diseño de Alto Nivel de los Módulos.....	11
4.1 R1B(Interfaz de usuario UI).....	11
4.2 R1C(Gestor de despliegue de Slices / Slice Manager)	12
4.3 R2(Soporte de cluster Linux).....	13
4.4 R3(Soporte de Openstack).....	14
4.4 R4(VM Placement).....	14
4.5 R5(Networking y Seguridad).....	15
5. Plan de Pruebas	17
5.1. Módulo 1: User interface.....	17
5.1.1 Autenticación de usuario	17
5.1.2 Visualización de template disponibles	18
5.1.3 Creación de un nuevo template.....	19
5.1.4 Eliminación de template	20
5.1.5 Edición de template	22
5.2. Módulo 2: Slice Manager	23
5.2.1 Validación de estructura del JSON.....	23
5.2.2 Selección de infraestructura	25
5.3. Módulo 3: Linux Driver.....	27
5.3.1 Creación de VMs en Linux Cluster	27
5.3.2 Conectividad entre VMs del mismo slice	28
5.3.3 Eliminación de Slice	30
5.4. Módulo 4: Openstack Driver	32
5.4.1 Creación de VMs	32
5.4.2 Conectividad entre VMs del mismo Slice	33
5.4.3 Eliminación de Slice	34
5.4.4 Obtención de URL para acceso de VNC a instancias.....	35
5.5. Módulo 5: VM Placement	36
5.5.1 Selección del host valida dentro de una Availability Zone.....	36
5.5.2 Selección correcta del host según la infraestructura elegida	38

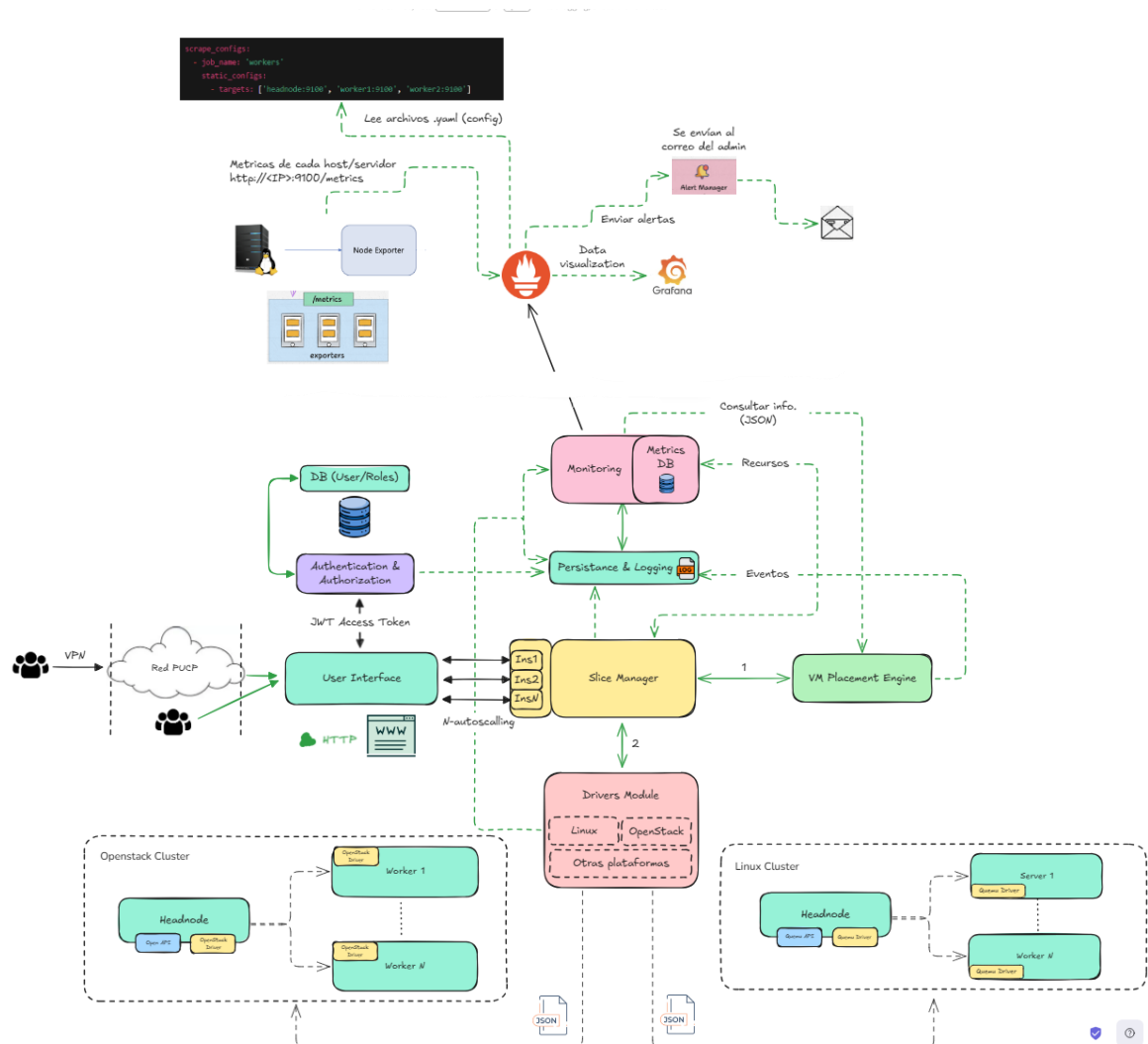
5.5.3 Rechazo de asignación por insuficiencia de recursos.....	40
5.6. Módulo 6: Persistence & Logging.....	43
5.6.1 Persistencia del estado del Slice en la Base de Datos	43
5.6.2 Registro de Logs al Crear/Eliminar slice	45
5.7. Módulo 7: Monitoreo	46
5.7.1 Disponibilidad de Node Exporter en los nodos del clúster	46
5.7.2 Validación de la recolección de métricas por Prometheus.....	48
5.7.3 Validación de la API de Monitoreo.....	50
5.7.4 Integración del Monitoreo con el Módulo de VM Placement	52

1.Introducción

El presente proyecto, denominado PUCP Private Cloud Orchestrator, tiene como objetivo el diseño e implementación de una plataforma integral que permita automatizar el ciclo de vida de máquinas virtuales y topologías de red en un entorno de nube privada, utilizando tanto infraestructura basada tanto en Linux como en OpenStack. Esta solución busca optimizar la administración de recursos dentro del clúster de servidores ubicados en la Pontificia Universidad Católica del Perú (PUCP).

El orquestador propuesto integra módulos de UI(Interfaz de Usuario), autenticación y autorización, gestión de slices, asignación de recursos (VM Placement), monitoreo de rendimiento, y red/seguridad, ofreciendo una interfaz accesible al usuario mediante servicios web (HTTP).

2.Arquitectura



La arquitectura del PUCP Private Cloud Orchestrator se basa en un conjunto de módulos interconectados que permiten automatizar el ciclo completo de vida de los slices, desde la solicitud del usuario hasta la creación, configuración y monitoreo de las máquinas virtuales, sobre entornos Linux y OpenStack.

El diseño propuesto sigue un enfoque modular y escalable, integrando componentes como el User Interface (Página Web + Manejo de sesiones), Slice Manager, Drivers Module, VM Placement Engine y Monitoring, los cuales interactúan mediante un servicio de colas de mensajes (RabbitMQ) y APIs REST (HTTP). Cada módulo y su funcionamiento se describen en detalle en las secciones siguientes.

3. Descripción de módulos

3.1 Módulo User Interface

El módulo User Interface constituye la capa de entrada del sistema y proporciona una interfaz web amigable para la interacción del usuario. Además de ofrecer un entorno de uso intuitivo, este módulo gestiona la parte del backend relacionada con la recepción de solicitudes, autenticación, autorización y persistencia de la información. Su función es garantizar que cada acción realizada en la interfaz sea autenticada mediante JWT Access Tokens, validada según los permisos del usuario y correctamente registrada en la base de datos.

A través de esta capa, los usuarios pueden crear, modificar y eliminar slices y plantillas, así como consultar su estado o historial de actividades. Estas operaciones se realizan mediante acciones CRUD sobre la base de datos del sistema, donde se mantienen los registros de usuarios, slices, templates y las transacciones asociadas al ciclo de vida del servicio. De esta forma, el User Interface actúa como la capa encargada de preservar la integridad y consistencia de la información almacenada.

Otro aspecto fundamental de este módulo es la generación del formato JSON estandarizado que encapsula cada solicitud antes de ser enviada al Slice Manager. La responsabilidad del User Interface finaliza precisamente en este punto: una vez que transforma la acción del usuario en una solicitud formal, la gestión del procesamiento interno deja de ser parte de sus funciones.

En términos de arquitectura, el User Interface coordina sus contenedores de servicios y APIs únicamente para facilitar la presentación de información y la persistencia de datos. No toma decisiones sobre la operación de los slices ni interviene en la asignación de recursos, sino que actúa como un puente confiable entre el usuario, el ecosistema backend y la base de datos. En síntesis, se trata de un módulo orientado a recibir, validar y registrar las acciones del usuario, asegurando que el estado del sistema se refleje de manera coherente y actualizada.

3.1.1 Conexión User Interface - Slice Manager

La interacción entre el User Interface y el Slice Manager comienza tan pronto como una acción del usuario es validada, autenticada y transformada por el User Interface en una solicitud formal. Una vez generado el JSON estandarizado que encapsula la operación deseada (creación, modificación, eliminación o consulta de un slice o

plantilla), el User Interface transfiere esta solicitud al Slice Manager, dando inicio al procesamiento interno del orquestador.

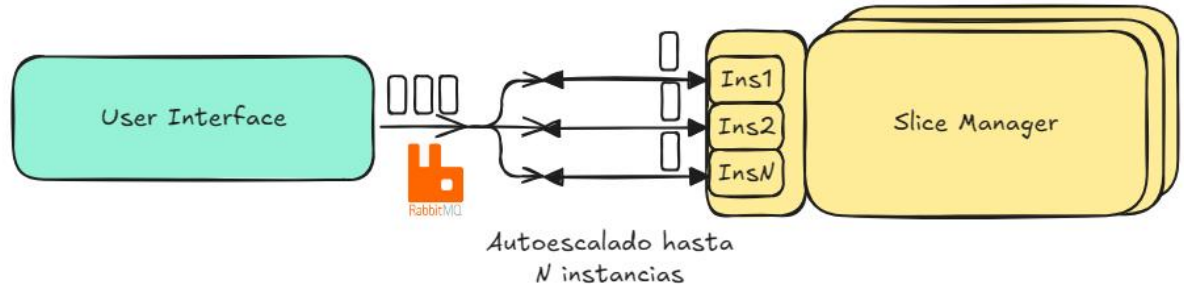
Cuando el sistema alcanza el número máximo de instancias permitidas por el autoescalado, las solicitudes adicionales no se pierden ni se rechazan: simplemente pasan a la cola manejada por RabbitMQ. Estas solicitudes en espera no serán procesadas inmediatamente, sino que quedarán retenidas de forma ordenada hasta que una instancia del Slice Manager termine su tarea.

En ese momento, la instancia liberada toma automáticamente la siguiente solicitud en cola, preservando el orden de llegada y garantizando que cada petición sea atendida sin interrupciones.

Es importante destacar que RabbitMQ no se activa únicamente en condiciones críticas. La cola está siempre presente como parte natural del flujo de procesamiento: absorbe picos de carga, coordina el despacho ordenado de solicitudes y proporciona mecanismos de reintento, confirmación y tolerancia a fallos, incluso bajo cargas normales.

Gracias a este diseño, el sistema mantiene un comportamiento estable, predecible y consistente, asegurando que cada solicitud sea eventualmente procesada aun cuando el número máximo de instancias del Slice Manager ya esté en uso.

Módulo API Gateway - Slice Manager:



3.2 Módulo Slice Manager

El Slice Manager constituye el núcleo lógico del sistema y es el encargado de ejecutar todo el procesamiento asociado al ciclo de vida de los slices. A diferencia del User Interface, que únicamente transforma las acciones del usuario en solicitudes formales, el Slice Manager interpreta, transforma y adapta dichas solicitudes al formato requerido por cada módulo interno. Esta adaptación incluye la normalización de parámetros, la traducción de estructuras y la generación de los modelos que otros servicios auxiliares necesitan para operar correctamente.

Su funcionalidad abarca el desglose de las peticiones recibidas, la validación de sus requerimientos y la determinación del perfil de consumo o reglas aplicables a cada caso. A partir de ello, coordina la interacción con los distintos componentes del

orquestador, como por ejemplo el módulo VM Placement, encargado de la asignación de recursos físicos o virtuales necesarios para la creación del slice.

Cada solicitud es gestionada dentro de una instancia independiente del Slice Manager, lo que permite que múltiples peticiones se procesen de manera simultánea sin interferencias entre sí. Esta arquitectura basada en instancias aisladas proporciona eficiencia y escalabilidad, garantizando que los ciclos de vida de distintos slices se desarrollen de forma autónoma. Según la lógica de operación requerida, el Slice Manager puede retener temporalmente una solicitud, subdividirla en etapas sucesivas, adaptar sus formatos internos o coordinar varias operaciones que deben ejecutarse en secuencia o en paralelo.

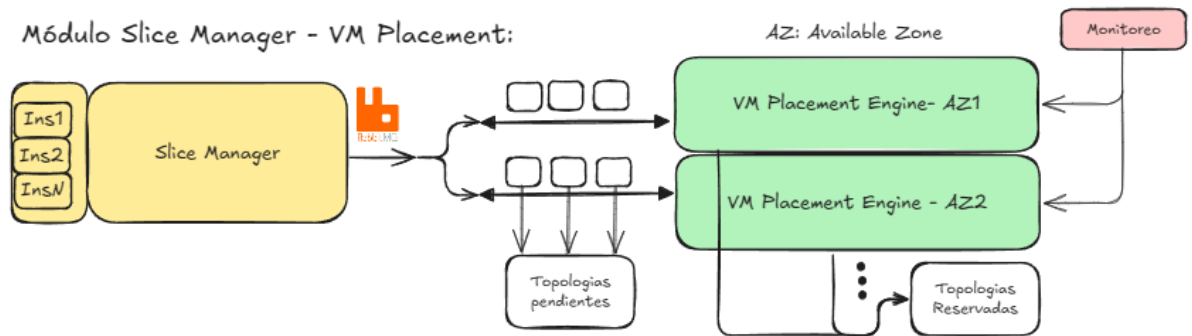
Durante toda la ejecución, este módulo también tiene la responsabilidad de notificar al User Interface sobre cada cambio relevante en el estado de un slice. Gracias a esta comunicación continua, la base de datos puede mantenerse actualizada y el usuario visualiza en tiempo real el avance de sus solicitudes. En conjunto, este flujo refuerza la separación funcional entre ambos módulos: mientras el User Interface se limita a presentar y registrar información, el Slice Manager actúa como el verdadero orquestador del sistema, encargado de procesar, coordinar y completar todas las operaciones necesarias para satisfacer cada petición desde su inicio hasta su finalización.

3.2.1 Conexión Slice Manager - VM Placement

El módulo de VM Placement se encuentra dividido en zonas de disponibilidad (AZs) con el objetivo de aislar los procesos de asignación de recursos entre ellas, evitando que una sobrecarga o falla en una zona afecte el desempeño de las demás. Este aislamiento se logra gracias a la independencia de los servidores en cada AZ, ya que ninguna comparte infraestructura física o lógica con otra, permitiendo así que cada zona sea tratada y gestionada de manera completamente autónoma.

Cada VM Placement Engine opera dentro de su respectiva AZ, gestionando sus propios recursos y topologías. De esta forma, mantiene un control independiente sobre las reservas pendientes, los recursos disponibles y las topologías asignadas.

Además, las instancias del VM Placement, separadas por AZ, son organizadas y coordinadas mediante RabbitMQ, que distribuye las solicitudes hacia la AZ correspondiente y garantiza que cada engine reciba las peticiones en el orden adecuado. Gracias a esta capa de coordinación, el Slice Manager puede interactuar con múltiples Placement Engines de forma ordenada y confiable sin generar interferencias entre zonas.



3.3 Módulo de VM Placement

El módulo de VM Placement implementa un mecanismo inteligente de asignación de recursos para un entorno de cómputo multiusuario donde las cargas de trabajo son altamente variables y la congestión debe mantenerse bajo control. Su objetivo es garantizar seguridad, eficiencia y resiliencia del clúster, combinando modelos probabilísticos con una estrategia de optimización global.

El sistema opera como un proceso de decisión en dos fases:

1. Filtro de Viabilidad (Seguridad):

Antes de evaluar la eficiencia, cada servidor es sometido a un análisis de riesgo mediante el cálculo de la Demanda Total Máxima Proyectada (DTMP).

- Este valor estima, de forma probabilística, la carga máxima que podría alcanzar un servidor considerando su estado actual y el nuevo slice.
- Solo los servidores cuya capacidad instalada sea suficiente para cubrir esta demanda se consideran candidatos.
Esto garantiza que ningún servidor sea seleccionado si existe un riesgo relevante de congestión.

2. Métrica de Selección (Balanceo de Riesgo):

Entre los servidores viables, el sistema aplica una estrategia Minimax que elige la opción que minimiza la máxima probabilidad de congestión en todo el clúster.

- Se evalúa el riesgo dominante por servidor (el recurso más vulnerable entre CPU y RAM).
- Luego se mide cómo afectaría esa asignación al riesgo global.
- La decisión final es aquella que balancea adecuadamente el riesgo extremo en el sistema.

Este enfoque permite mantener la estabilidad del clúster en el largo plazo, evitando puntos críticos de fallo y logrando un uso eficiente de los recursos.

El modelo toma en cuenta características reales de uso mediante un perfil de usuario y un contexto técnico, lo que permite estimar de manera más precisa el consumo promedio y la variabilidad de CPU y RAM. Además, integra mecanismos de retroalimentación basados en métricas reales obtenidas del sistema de monitoreo, mitigando efectos indeseados como el **cold start**.

Finalmente, el módulo incorpora restricciones deterministas, como la capacidad de disco y la atomicidad del despliegue por slice, para asegurar que las asignaciones cumplan con las condiciones operativas del entorno.

En conjunto, el módulo implementa un esquema de asignación segura, adaptativa y globalmente balanceada, alineado con las necesidades de plataformas modernas de virtualización y orquestación.

Para un mayor nivel de detalle técnico sobre los fundamentos, modelos probabilísticos, ecuaciones, supuestos y justificaciones del proceso de asignación, se ha elaborado un documento informativo complementario. Este material amplía la explicación del filtro de viabilidad (DTMP), la estrategia Minimax, los modelos de consumo por perfil y contexto técnico, así como el análisis de riesgo a nivel de clúster.

3.3.1 Coordinación Slice Manager - VM Placement

La interacción entre el Slice Manager y el VM Placement se realiza a través de un flujo estructurado basado en mensajes JSON y APIs REST, garantizando una comunicación eficiente y estandarizada.

Inicialmente, el Slice Manager recibe una solicitud de creación de slice desde la interfaz de usuario, la cual se encuentra codificada en formato JSON e incluye los requerimientos de recursos tales como CPU, memoria RAM, espacio en disco, la zona de disponibilidad deseada, la plataforma de despliegue (Linux u OpenStack), así como el perfil y contexto técnico del usuario.

Dado que el VM Placement opera sobre un modelo probabilístico específico y únicamente requiere información esencial para la asignación de recursos, el Slice Manager se encarga de extraer de la solicitud original únicamente los parámetros estrictamente necesarios, tales como la zona de disponibilidad y los recursos solicitados (CPU, RAM, disco). Esta transformación asegura que el VM Placement reciba un conjunto de datos minimalista y relevante, manteniendo el desacoplamiento entre módulos y evitando que información innecesaria, como detalles de topología o conexiones, afecte la operación del motor de placement ni interfiera con la capa superior del sistema.

Una vez realizada esta adaptación, el Slice Manager transmite la solicitud al VM Placement a través de las APIs REST correspondientes. El motor de placement evalúa la viabilidad del slice mediante su algoritmo, y tras completar la evaluación,

devuelve un JSON con el resultado, indicando si el slice puede ser aceptado o rechazado. En caso de aceptación, la respuesta incluye, además del worker asignado, métricas internas relevantes, como la probabilidad de congestión.

Finalmente, el Slice Manager deriva la solicitud al módulo Drivers, encargado de la creación efectiva de las máquinas virtuales en la plataforma física correspondiente. Este flujo asegura una coordinación clara, desacoplada y estandarizada entre los módulos, manteniendo consistencia en la asignación de recursos y eficiencia en los despliegues.

3.4 Módulo de Drivers

Una vez que el algoritmo de VM Placement determina en qué infraestructura y en qué host debe ubicarse cada VM, el Slice Manager genera un plan de despliegue final y lo envía al Módulo de Drivers. Este módulo recibe dicho plan en formato JSON ya resuelto y se encarga de traducirlo a operaciones concretas en la infraestructura correspondiente.

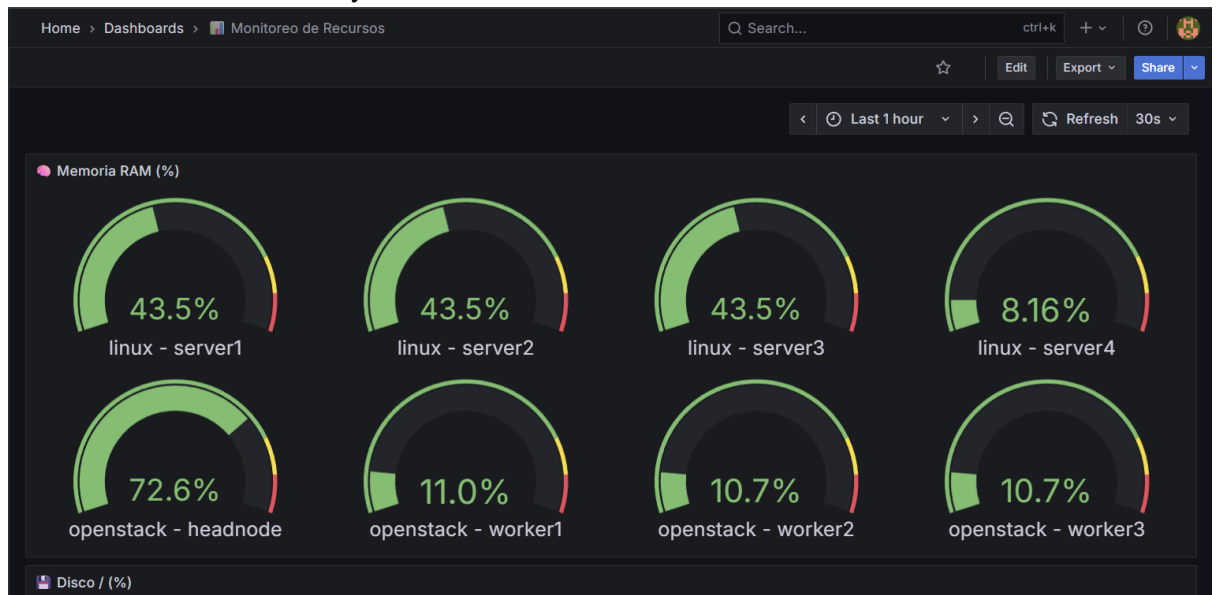
- **Linux Driver:** Cuando el JSON indica que una VM va al clúster Linux, el driver convierte el plan en llamadas internas mediante APIs para crear imágenes qcow2, lanzar QEMU/KVM en el worker especificado, crear interfaces TAP y conectarlas a OVS con la VLAN indicada. Todo se ejecuta usando la información ya resuelta por VM Placement (worker destino, cantidad de NICs, VLANs y configuración). El driver retorna al Slice Manager los identificadores generados (VM, TAPs, VNC, VLAN).
- **Openstack Driver:** Cuando el plan indica que la VM debe desplegarse en OpenStack, el driver traduce esa sección del JSON a llamadas directas al SDK de OpenStack (Keystone, Neutron, Nova). Con esto crea el proyecto, redes, puertos y la instancia Nova en el hypervisor asignado por VM Placement, respetando la cantidad de NICs, subredes y configuraciones definidas en la topología. Finalmente, retorna los IDs de red, puertos y servidor creada.

3.4 Módulo de Monitoreo

El Módulo de Monitoreo se encarga de recolectar información actualizada sobre el estado de cada infraestructura, tanto Linux como OpenStack, para que el orquestador pueda tomar decisiones informadas. Para ello realiza consultas periódicas mediante APIs.

Toda la información recogida se estructura en formato JSON, manteniendo un modelo unificado independientemente del backend. Ese JSON es enviado al Módulo de VM Placement, que utiliza estos datos para evaluar disponibilidad, carga y restricciones definidas por zonas de disponibilidad y, en base a ello, decidir dónde debe ubicarse cada VM del slice. En este módulo nos apoyamos del aplicativo

Grafana, donde podemos ver en tiempo real el consumo de los workers y servers, lo cual extrae métricas con ayuda de Prometheus.



3.5 Módulo de Persistence & Logging

El Módulo de Persistencia & Logging es responsable de almacenar de manera permanente toda la información relevante de los slices y registrar el historial de operaciones del orquestador. Este módulo utiliza una base de datos SQL donde se guardan:

- Los parámetros esenciales de cada slice: topología, VMs creadas, redes, infraestructura asignada, IDs devueltos por los drivers, zonas de disponibilidad y estado actual.
- Los estados intermedios y finales del ciclo de vida del slice (creación, despliegue, eliminación).
- Los logs detallados de cada operación emitida por el Slice Manager y por los drivers (Linux y OpenStack), incluyendo errores, tiempos de ejecución y recursos creados o liberados.

Este módulo permite que el orquestador mantenga coherencia entre lo desplegado y lo almacenado, facilita auditoría, depuración y recuperación de estado, y abastece a la UI con información consistente incluso si los procesos de despliegue ya finalizaron.

4. Diseño de Alto Nivel de los Módulos

En esta sección se presenta el Diseño de Alto Nivel (High-Level Design, HLD) de los principales módulos que conforman el PUCP Private Cloud Orchestrator, desarrollado en cumplimiento de los requerimientos del proyecto.

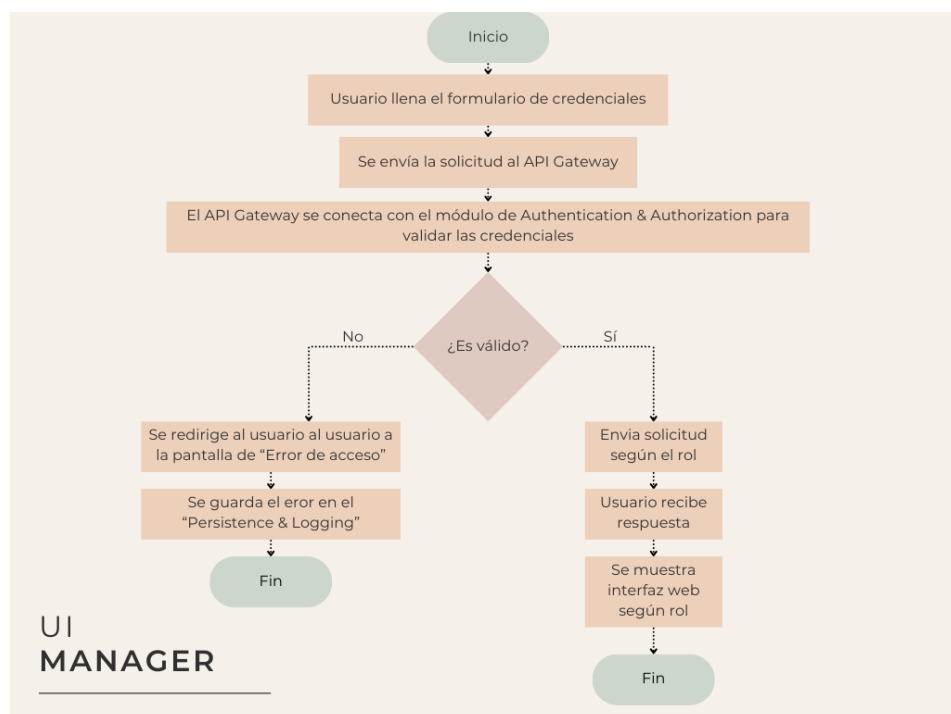
El objetivo de este diseño es describir la estructura lógica y funcional de cada módulo, detallando cómo interactúan entre sí para cumplir el ciclo completo de despliegue y gestión de slices dentro del orquestador.

La arquitectura propuesta se basa en un enfoque modular y escalable, donde cada componente asume responsabilidades claramente definidas: desde la interfaz con el usuario hasta la configuración de red y seguridad en los entornos virtualizados.

4.1 R1B(Interfaz de usuario UI)

La Interfaz de Usuario constituye la entrada al sistema. Su función es permitir que el usuario cree, modifique y elimine slices mediante una interfaz web intuitiva. Este módulo:

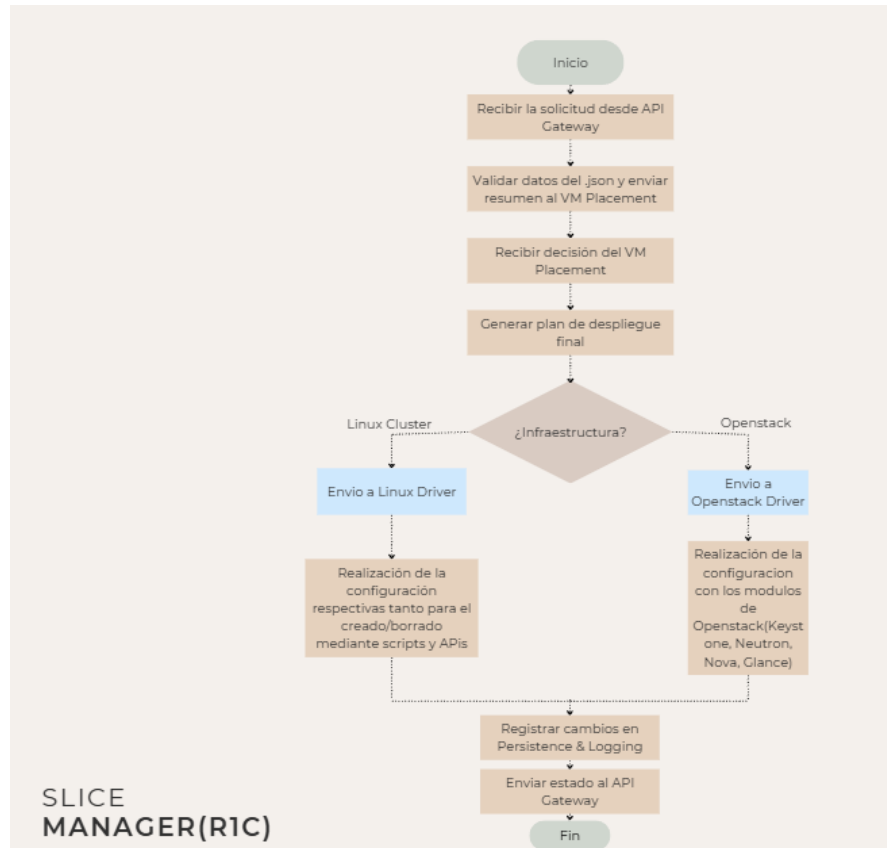
- Gestiona autenticación y autorización mediante JWT.
- Valida las acciones permitidas según el rol del usuario.
- Gestiona los templates generados.
- Registra las operaciones en la base de datos.
- Genera el JSON estandarizado que describe la solicitud.
- Envía dicho JSON al Slice Manager y espera actualizaciones de estado.



4.2 R1C(Gestor de despliegue de Slices / Slice Manager)

El Slice Manager es el núcleo lógico del orquestador. Recibe las solicitudes generadas por la UI y coordina el ciclo de vida completo de cada slice. Sus responsabilidades son:

- Recibir la solicitud JSON y enviarla al VM Placement.
- Coordinar con el módulo de VM Placement, enviando únicamente los recursos necesarios.
- Invocar al Módulo de Drivers para materializar el despliegue en Linux u OpenStack.
- Reportar estados y errores hacia la UI y Persistencia & Logging.



4.3 R2(Soporte de cluster Linux)

Este driver opera exclusivamente sobre el Linux Cluster, donde las VMs se crean mediante QEMU/KVM sobre servers Linux.

Recibe desde el Slice Manager:

- Worker asignado (server2-4)
- Imagen base
- Cantidad de NICs
- VLANs
- Recursos de CPU, RAM, Disco

Acciones del driver:

- Crear imágenes .qcow2 derivadas.
- Lanzar VMs con QEMU/KVM en el worker asignado.
- Crear interfaces TAP por cada NIC.

- Asociar TAPs a Open vSwitch (OVS) con sus VLAN respectivas.
Ejecutar los archivos de deploy del Linux Cluster, ubicados en server4, que configuran:
 - Bridges OVS
 - Reglas de NAT
 - VLAN tagging
 - Limitación de broadcast
- Retornar IDs y estados al Slice Manager.

4.4 R3(Soporte de Openstack)

Este driver opera sobre el OpenStack HeadNode, utilizando el SDK para crear recursos en el clúster OpenStack.

Recibe desde el Slice Manager:

- Hypervisor OpenStack asignado
- Imagen (Glance)
- Flavor
- NICs, redes y subredes definidas
- Zonas de disponibilidad

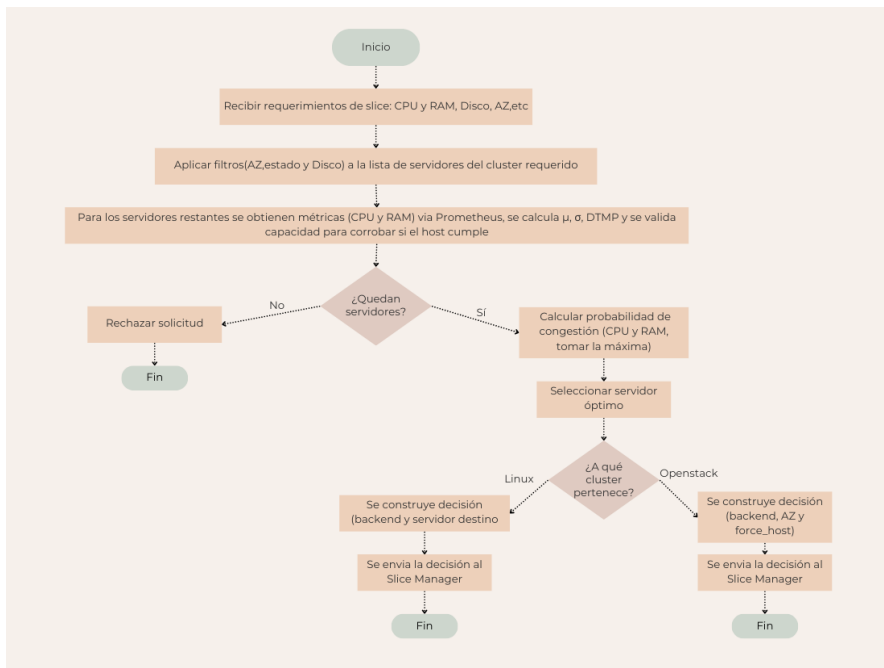
Acciones del driver:

- Crear/Usar el proyecto correspondiente al slice.
- Crear redes, subredes y puertos mediante Neutron.
- Crear las instancias Nova con las NICs definidas.
- Retornar IDs reales de redes, puertos y VMs al Slice Manager.

4.4 R4(VM Placement)

El Módulo de VM Placement es el encargado de decidir en qué infraestructura (Linux Cluster u OpenStack Cluster) y en qué servidor debe desplegarse cada VM de un slice. Para ello utiliza un conjunto reducido de parámetros enviados por el Slice Manager (CPU, RAM, disco y AZ) y las métricas de carga actuales proporcionadas por el Módulo de Monitoreo.

El resultado es un JSON final enviado al Slice Manager que indica la infraestructura elegida (Linux u OpenStack), el worker/hypervisor asignado y el estado de la evaluación. Este JSON sirve como base para que los Drivers ejecuten el despliegue real.



4.5 R5(Networking y Seguridad)

El módulo de Networking & Seguridad aplica la configuración de red según el cluster asignado. Su responsabilidad es definir reglas, aislamiento y conectividad.

Toda la configuración de red del Linux Cluster se realiza mediante archivos de deploy ubicados en server4, ejecutados automáticamente por el driver.

Incluye:

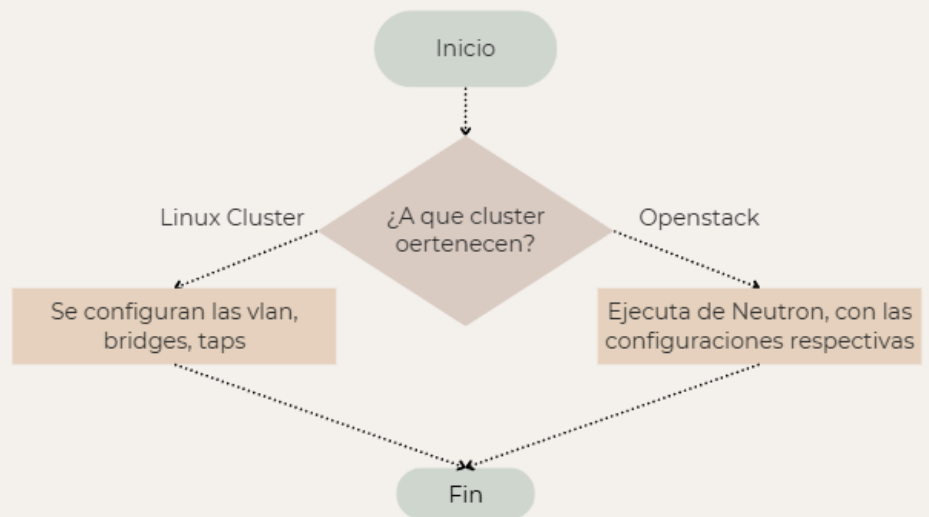
- Creación de bridges OVS (br-int, br-ofs, etc.).
- Etiquetado de VLANs para cada tap.
- Configuración de NAT y reglas iptables para dar acceso externo.

El módulo solo define la política; la ejecución ocurre en server4.

La configuración del entorno OpenStack se ejecuta desde el HeadNode, usando:

- SDK de Neutron
- Reglas de Security Groups
- Configuración de routers internos y externos
- Topologías de red definidas para el slice

El módulo establece la política de aislamiento y las reglas, pero la ejecución se realiza mediante los scripts de deploy en el HeadNode y las APIs de OpenStack.



NETWORKING & SECURITY MANAGER(R5)